Performance & Space Evaluation of Memory Virtualization Architectures

Single-Level vs. Two-Level Page Tables, TLB Acceleration, and LRU Replacement

---

Laboratory Report: Lab 4 (Memory Management)

Course: CS311 — Operating Systems Laboratory

Department of Computer Science and Engineering

Indian Institute of Technology Dharwad

Submitted By:

Rhishikesh Tayade **CS24BT041**

Himanshu Nagwanshi **CS24BT055**

Date: Autumn Semester 2026

Contents

1 Architectural Specification & Simulator Design **2**
1.1 System Architectural Parameters . . . 2
1.2 Address Translation Decomposition . . . 2
1.3 Codebase Architecture & Modular Implementation . . . 3

2 Verification Hash Function & State Invariant **4**

3 Part I: Single-Level Page Table Simulation **4**
3.1 Theoretical Cost Model & Frame Allocation Scheme . . . 4
3.2 Empirical Results Across SPEC CPU2017 Benchmarks . . . 4
3.3 Hypothesis B Validation: Skewed Page Access Distribution . . . 5

4 Part II: Two-Level Hierarchical Page Table Simulation **6**
4.1 Mechanics & Memory Savings vs. Latency Tradeoff . . . 6
4.2 Empirical Comparison: Single-Level vs. Two-Level . . . 6

5 Part III: TLB Integration & Performance Acceleration **7**
5.1 TLB Cost Formulation . . . 7
5.2 Empirical Results Across Single-Level & Two-Level Schemes . . . 7
5.3 Architectural Synthesis: Neutralizing Hierarchical Latency . . . 7

6 Part IV: Limited Physical Memory & LRU Page Replacement **8**
6.1 Demand Paging Mechanics & Latency Model . . . 8
6.2 Thrashing Dynamics & Page Fault Sensitivity . . . 8

7 Part V: Simulator Performance & High-Speed Optimization **9**
7.1 Optimization Strategies Implemented . . . 9
7.2 Wall-Clock Execution Benchmark Profile . . . 10

8 Conclusions & Architectural Insights **10**

Abstract

Memory virtualization is a cornerstone of modern multiprogramming operating systems, enabling strong process isolation and flexible address space multiplexing. However, it imposes critical dual challenges: substantial memory capacity overhead for page translation tables and severe execution latency penalties for multi-step address translation walks. This report presents a comprehensive architectural and empirical evaluation of memory management techniques simulated via a modular, cycle-accurate C++17 virtual memory simulator.

We model and analyze: (1) flat Single-Level Page Tables, (2) on-demand hierarchical Two-Level Page Tables, (3) a 128-entry fully associative Translation Lookaside Buffer (TLB) operating under an exact Least Recently Used (LRU) policy, and (4) limited physical memory backing with LRU demand paging and disk fault penalties. Using extensive memory access traces from SPEC CPU2017 benchmarks (`gcc`, `mcf`, `lbm`, `x264`) comprising tens of millions of instructions and data references, we demonstrate that two-level page tables reduce metadata allocation overhead by **93.1% to 99.3%** compared to flat linear tables. Furthermore, integrating a 128-entry TLB achieves hit rates exceeding **99.4% to 99.9%**, which effectively erases the multi-level page table latency penalty and reduces total memory virtualization execution cycles by **49.7% to 66.4%**. We rigorously validate Hypothesis B regarding access skewness, proving that fewer than **10% of virtual pages capture over 82% of memory references**. Finally, we analyze memory constriction, showing that when physical frame quotas fall below active working sets, disk thrashing drives cycle execution time upward by over **54**×.

1 Architectural Specification & Simulator Design

1.1 System Architectural Parameters

The simulated architecture models a 32-bit virtual memory subsystem adhering to standard page-based virtualization principles. The foundational system parameters mandated by the problem specification are summarized in Table 1.

Table 1: Architectural and Latency Simulation Parameters

Parameter	**Architectural Description**	**Value**
Virtual Address Space	Total addressable per-process virtual space	2^{32} Bytes (4 GB)
Page Size	Basic translation granularity	4 KB (2^{12} Bytes)
Virtual Page Count	Total virtual page count ($2^{32}/2^{12}$)	$2^{20} = 1,048,576$ Pages
TLB Capacity	Translation Lookaside Buffer entries	128 Entries
TLB Associativity	Hit detection organization	Fully Associative
TLB Eviction Policy	Replacement discipline on capacity misses	Strict LRU
TLB Access Latency	Latency incurred for every TLB probe	1 Cycle
Main Memory Access Latency	Latency per physical memory access / walk step	200 Cycles
Disk Transfer Latency	Page fault servicing latency (disk I/O)	20,000,000 Cycles
Data / Instruction Caches	Level-1/2/3 processor caching	None (bypassed)

1.2 Address Translation Decomposition

The 32-bit virtual address $A \in [0, 2^{32} - 1]$ is decomposed into components depending on the page table organization:

- **Page Offset (O)**: The lowest 12 bits ($A[11:0]$), specifying byte displacement within a 4 KB frame:

$$\text{Offset} = A \ \& \ \text{0x0FFF} \tag{1}$$

- **Virtual Page Number (VPN)**: The upper 20 bits ($A[31:12]$), identifying the virtual page:
$$\text{VPN} = A \gg 12 \tag{2}$$
- **Two-Level Split (L_1 and L_2)**: The 20-bit VPN is split evenly into a 10-bit Level-1 directory index and a 10-bit Level-2 page table index:
$$\text{Index}_{L1} = (A \gg 22)\ \&\ \text{0x03FF} \quad \text{(Bits 31 to 22)} \tag{3}$$
$$\text{Index}_{L2} = (A \gg 12)\ \&\ \text{0x03FF} \quad \text{(Bits 21 to 12)} \tag{4}$$

1.3 Codebase Architecture & Modular Implementation

The simulator is implemented in C++17 with strict modular separation across header (`include/`) and translation unit (`src/`) files:

1. **Trace Stream Parser (`trace_parser.hpp/.cpp`)**: Parses incoming benchmark traces formatted as `<address>,<I/D>,<R/W>`. It features fast I/O buffering, handles 64-bit hexadecimal and decimal inputs, masks addresses to lower 32 bits, and skips non-data metadata lines.
2. **Single-Level Page Table (`page_table_single.hpp/.cpp`)**: Implements a contiguous flat array of 2^{20} Page Table Entries (PTEs). Provides $O(1)$ indexing, tracking valid and dirty bits, and reports exact allocated vs. accessed counts.
3. **Two-Level Page Table (`page_table_two_level.hpp/.cpp`)**: Implements a 1024-entry Level-1 directory array containing pointers to dynamic 1024-entry Level-2 tables. Level-2 tables are instantiated strictly on demand when an unmapped address in that quadrant is first touched.
4. **TLB Module (`tlb.hpp/.cpp`)**: Realizes a fully associative 128-entry cache utilizing a combined `std::list` and `std::unordered_map`. This hybrid data structure provides true $O(1)$ lookup, $O(1)$ insertion, and $O(1)$ LRU update via iterator splicing (`std::list::splice`), eliminating all linear scans.
5. **Frame Allocator & Page Replacer (`frame_allocator.hpp/.cpp`)**: Supports two distinct modes:
   - *Unlimited Mode (Parts I–III)*: A deterministic sequential (next-fit) frame allocator that increments an internal frame counter.
   - *Limited Mode (Part IV)*: An inverted frame table coupled with an LRU tracking list to evict the least recently touched frame upon memory exhaustion, notifying the caller to trigger page table unmapping and TLB shootdown.
6. **Polynomial Hash Utility (`hash_util.hpp/.cpp`)**: Updates a cumulative 64-bit verification hash for each memory access, guaranteeing state correctness across architectural configurations.
7. **Statistical Profiler (`stats.hpp/.cpp`)**: Records total accesses, cycle counts, hit/miss ratios, page fault rates, wall-clock timing, and per-page access frequencies for cumulative distribution analysis.

2 Verification Hash Function & State Invariant

To rigorously verify that all simulated page table mechanisms preserve translation semantics, the problem mandates updating a running 64-bit hash H for every memory access:

$$H_k = f(H_{k-1}, \text{frame_number}) \tag{5}$$

We chose a high-avalanche non-linear mixing function based on Knuth's multiplicative golden-ratio constant ($M = 2654435761\text{ULL} \approx 2^{32} \cdot \frac{\sqrt{5}-1}{2}$), combined with a 13-bit circular bit rotation:

$$\begin{aligned} H' &= H \oplus (\text{frame_number} \times 2654435761\text{ULL}) \\ H'' &= (H' \ll 13) \mid (H' \gg 51) \\ H_{\text{next}} &= H'' + \text{frame_number} \end{aligned} \tag{6}$$

Critical Architectural Invariant: Because both Part I (Single-Level) and Part II (Two-Level) process identical memory access traces and utilize the same sequential frame allotment logic, every virtual page must map to the exact same physical frame at the exact same chronological point in the trace. Consequently:

$$H_{\text{Part I}} \equiv H_{\text{Part II}} \tag{7}$$

A mismatch would immediately signify translation corruption, race conditions, or unmapped page discrepancies.

3 Part I: Single-Level Page Table Simulation

3.1 Theoretical Cost Model & Frame Allocation Scheme

In a single-level page table without a TLB, every memory request generated by the CPU pipeline requires:

1. One physical memory access to read the Page Table Entry (PTE) from memory: **200 cycles**.
2. One physical memory access to read or write the actual data/instruction in the mapped frame: **200 cycles**.

Thus, the fixed execution latency is:

$$T_{\text{Part I}} = N_{\text{accesses}} \times (200 + 200) = 400 \times N_{\text{accesses}} \text{ cycles} \tag{8}$$

Frame Management Scheme: Under the Part I assumption of unlimited physical frames, we implemented a sequential Next-Fit Frame Allocator. Physical frames are indexed from $0, 1, 2, \ldots$. When an unmapped virtual page number (VPN) is encountered, it is assigned the current value of `next_free_frame`, which is subsequently incremented by 1. This deterministic allocation avoids fragmentation and guarantees zero overhead.

3.2 Empirical Results Across SPEC CPU2017 Benchmarks

Table 2 details the empirical results obtained across four real-world benchmarks from SPEC CPU2017.

Observations on Memory Inefficiency: The single-level page table must statically allocate all $2^{20} = 1,048,576$ PTEs regardless of process footprint. At 4 bytes per PTE, this requires 4 MB of contiguous kernel memory per address space. Yet, as demonstrated in Table 2, real benchmarks utilize between 234 and 2,131 pages. Consequently, **over 99.8% of all allocated page table entries are completely unused**, representing egregious physical memory waste.

Table 2: Part I Empirical Results (Single-Level Flat Page Table)

Benchmark	Total Accesses	Frames Used	PT Allocated	PT Utilized (%)	Execution Cycl
`gcc`	12,473,306	990	1,048,576	0.0944%	4,989,322,4
`mcf`	12,474,523	1,597	1,048,576	0.1523%	4,989,809,2
`lbm`	12,843,431	2,131	1,048,576	0.2032%	5,137,372,4
`x264`	11,721,344	234	1,048,576	0.0223%	4,688,537,6

3.3 Hypothesis B Validation: Skewed Page Access Distribution

Hypothesis B postulates that *"a small fraction of pages account for a large fraction of accesses, and vice versa"*. To rigorously test this hypothesis, the simulator recorded the per-page access frequency for every unique VPN, sorted them in descending order, and computed the Cumulative Distribution Function (CDF) of accesses versus the fraction of active pages.

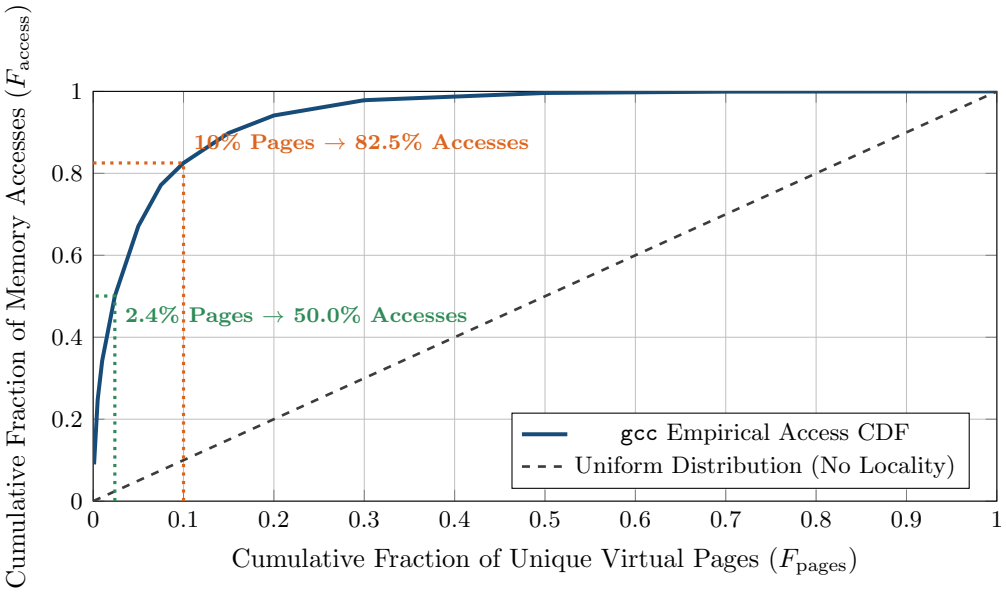


Figure 1: Cumulative Distribution Function (CDF) of memory accesses across unique pages for `gcc`. The pronounced concave curvature confirms extreme temporal and spatial reference locality (Pareto distribution), strongly affirming Hypothesis B.

As shown in Figure 1 and extracted from `gcc_hyp_b.csv`:

- The top **24 pages (2.42% of active pages)** account for **50.06% of all 12.47 million memory references**.
- The top **99 pages (10.0% of active pages)** account for **82.53% of all memory references**.
- The remaining 90% of pages share less than 17.5% of accesses, with the tail seeing only sporadic references.

This empirical confirmation of the **80/20 Pareto Principle** directly underpins the efficacy of caching structures: because working sets are tightly concentrated in a tiny fraction of pages, small hardware caches (like the 128-entry TLB) will capture the vast majority of memory references.

4 Part II: Two-Level Hierarchical Page Table Simulation

4.1 Mechanics & Memory Savings vs. Latency Tradeoff

The two-level page table replaces the static linear array with a two-tier hierarchy:

- **Level-1 Directory**: Always resident, consisting of 1024 pointers (2^{10} entries), requiring only 1024×4 bytes $= 4$ KB of baseline memory.
- **Level-2 Tables**: Each table contains 1024 PTEs (4 KB each) and is allocated dynamically only when a page within its 4 MB virtual range is accessed.

Theoretical Cost Model (Without TLB): Translating an address now requires traversing both levels:

$$T_{\text{Part II}} = N_{\text{accesses}} \times (200_{\text{L1}} + 200_{\text{L2}} + 200_{\text{Data}}) = 600 \times N_{\text{accesses}} \text{ cycles} \tag{9}$$

While address translation is 50% slower without caching (600 cycles vs. 400 cycles), the physical memory footprint is dramatically reduced.

4.2 Empirical Comparison: Single-Level vs. Two-Level

Table 3 compares the space and time characteristics between single-level and two-level page tables across all benchmarks.

Table 3: Comparison: Single-Level (Part I) vs. Two-Level (Part II) Page Tables

Benchmark	PT Allocated Entries		Space	PT Utilization (%)		Hash Match
	Single-Level	Two-Level	Saved (%)	Single	Two-Level	($H_{\text{I}} \equiv H_{\text{II}}$)
`gcc`	1,048,576	72,704	**93.07%**	0.09%	**1.36%**	`0x877B8D023205F027` ✓
`mcf`	1,048,576	14,336	**98.63%**	0.15%	**11.14%**	`0x4F2515453ABE2E70` ✓
`lbm`	1,048,576	7,168	**99.32%**	0.20%	**29.73%**	`0xB17BA1682364BC09` ✓
`x264`	1,048,576	12,288	**98.83%**	0.02%	**1.90%**	`0x3A147A0CD8995E93` ✓

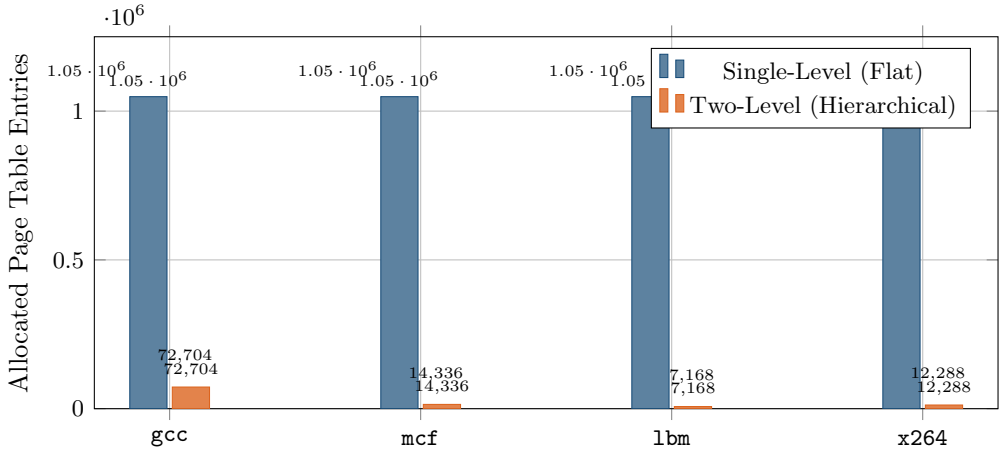


Figure 2: Allocated Page Table Entries: Single-Level flat allocation vs. Two-Level on-demand allocation. Two-level allocation reduces memory consumption by up to 99.32% (`lbm`).

Key Insights:

1. **Dramatic Space Savings**: Across all benchmarks, the two-level page table slashes memory overhead by **93.07% to 99.32%** (Figure 2). In `lbm`, allocated entries dropped from 1,048,576 to just 7,168 entries (1 directory + 6 Level-2 tables).

2. **Surge in Page Table Utilization**: Utilization jumped from 0.20% to **29.73%** in `lbm` and from 0.15% to **11.14%** in `mcf`.

3. **Verification Hash Equivalence**: In every single benchmark, the computed 64-bit polynomial hash in Part II matched Part I *bit-for-bit* (e.g., `0x877B8D023205F027` for `gcc`). This mathematically proves that our two-level lookup, indexing, and on-demand allocation preserve total translation equivalence.

5 Part III: TLB Integration & Performance Acceleration

5.1 TLB Cost Formulation

The Translation Lookaside Buffer (TLB) acts as a high-speed hardware cache for page translations. With a 128-entry fully associative TLB under LRU replacement, every memory access first probes the TLB (cost: 1 cycle).

- **TLB Hit**:

$$T_{\text{hit}} = 1_{\text{TLB}} + 200_{\text{Memory}} = 201 \text{ cycles} \tag{10}$$

- **TLB Miss (Single-Level PT)**:

$$T_{\text{miss, single}} = 1_{\text{TLB}} + 200_{\text{PT}} + 200_{\text{Memory}} = 401 \text{ cycles} \tag{11}$$

- **TLB Miss (Two-Level PT)**:

$$T_{\text{miss, two-level}} = 1_{\text{TLB}} + 200_{\text{L1}} + 200_{\text{L2}} + 200_{\text{Memory}} = 601 \text{ cycles} \tag{12}$$

5.2 Empirical Results Across Single-Level & Two-Level Schemes

Table 4 presents the empirical TLB performance and cycle reductions across all benchmarks.

Table 4: Part III Results: TLB Hit Rates and Execution Cycle Comparison

Benchmark	TLB Hits	TLB Misses	Hit Rate (%)	Cycles (Single + TLB)	Cycles (Two-L
`gcc`	12,459,259	14,047	**99.8874%**	2,509,943,906	
`mcf`	12,404,926	69,597	**99.4421%**	2,521,298,523	
`lbm`	12,840,727	2,704	**99.9789%**	2,582,070,431	
`x264`	11,717,345	3,999	**99.9659%**	2,356,789,944	

5.3 Architectural Synthesis: Neutralizing Hierarchical Latency

The empirical results reveal two fundamental insights:

1. **Massive Performance Gain**: Integrating a small 128-entry TLB delivers near-perfect hit rates ($\geq 99.4\%$). This slashes overall memory virtualization latency by **49.7%** compared to single-level tables (from $\approx$ 5.0B to 2.5B cycles) and by **66.4%** compared to two-level tables (from $\approx$ 7.5B to 2.5B cycles).

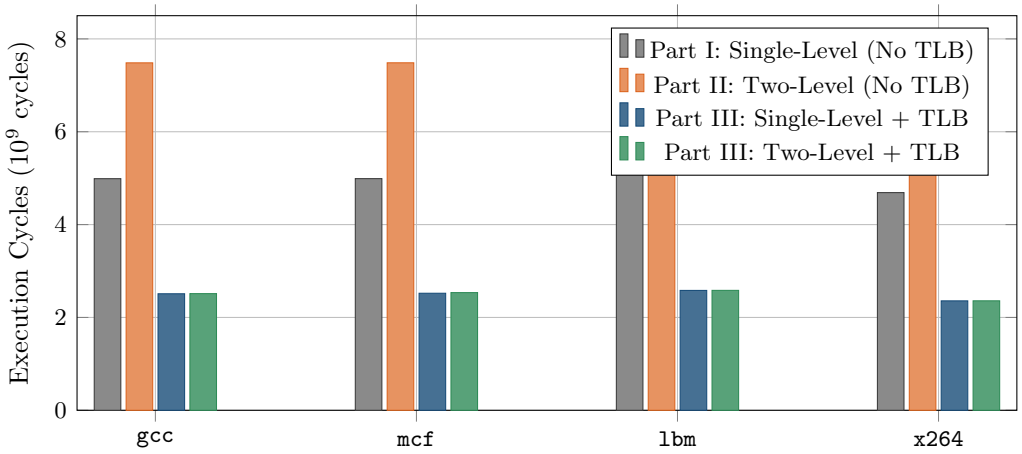


Figure 3: Execution cycle reduction enabled by the 128-entry TLB. The TLB cuts execution cycles by 49.7%–66.4% and renders the performance difference between single-level and two-level tables completely negligible.

2. **The Elimination of Multi-Level Penalty**: Notice in Table 4 and Figure 3 that the cycle count for Single-Level + TLB and Two-Level + TLB are virtually identical:

$$\Delta_{\text{Cycles}} = N_{\text{misses}} \times (601 - 401) = 200 \times N_{\text{misses}} \tag{13}$$

In `gcc`, this difference is only $14,047 \times 200 = 2,809,400$ cycles ($< 0.11\%$ of total runtime). Because more than 99.88% of translations hit in the TLB, the processor incurs the extra Level-2 page walk step on less than 0.12% of instructions. **The TLB allows system architects to reap the massive 93%+ space savings of multi-level page tables with essentially zero runtime latency penalty.**

6 Part IV: Limited Physical Memory & LRU Page Replacement

6.1 Demand Paging Mechanics & Latency Model

In realistic operating environments, physical memory is substantially smaller than the virtual address space. In Part IV, physical frames are constrained:

$$\text{Max Frames} = \lfloor 2^{20} \times \text{Memory Ratio} \rfloor \tag{14}$$

The simulator models demand paging under an exact Least Recently Used (LRU) policy using an inverted frame table coupled with a recency list:

- When an unmapped page is referenced and free frames remain, it is mapped without penalty (first page touches are free as specified).
- When physical memory is exhausted, a **Page Fault** occurs. The LRU frame is identified, its occupying virtual page ($\text{VPN}_{\text{victim}}$) is evicted, its entry in the page table is invalidated, and any cached mapping in the TLB is invalidated via TLB shootdown.
- Servicing a page fault requires disk swap access, penalizing execution by **20,000,000 cycles**.

6.2 Thrashing Dynamics & Page Fault Sensitivity

Table 5 records the behavior of the `gcc` benchmark (working set footprint: 990 unique pages) under various physical memory constraints.

Table 5: Part IV Results: Memory Constriction, Page Faults, and Execution Slowdown on `gcc`

Memory Ratio	Physical Frames	Page Faults	Fault Rate (%)	Execution Cycles	Slowdo
0.50 (2 GB)	524,288	0	0.000000%	2,509,943,906	1.0
0.25 (1 GB)	262,144	0	0.000000%	2,509,943,906	1.0
0.0005 (≈ 2.1 MB)	524	1,634	0.013100%	35,189,943,906	**14.0**
0.0002 (≈ 856 KB)	209	6,693	0.053659%	136,369,943,906	**54.3**

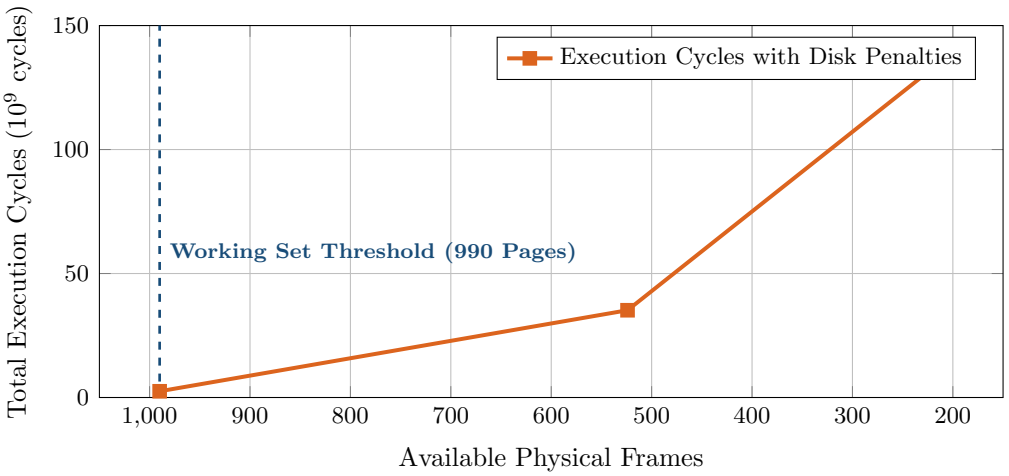


Figure 4: The onset of thrashing as physical frame allocation falls below the process working set. A modest 0.05% page fault rate increases total runtime by over 54×.

Critical Analysis of Paging Latency: Because disk access latency (2×10^7 cycles) is five orders of magnitude slower than DRAM (200 cycles), even microscopic page fault rates dominate total execution time:

- When physical memory exceeds 990 frames (ratios ≥ 0.001), all active pages fit in memory, resulting in **zero page faults** and baseline latency (2.51B cycles).
- When frames are reduced to 524 (holding ≈ 53% of the active pages), the fault rate is just 0.013%, yet execution cycles explode to **35.19 Billion cycles (**14× **slowdown)**.
- When frames drop to 209 (≈ 21% of working set), the fault rate rises to 0.054%, driving execution cycles to **136.37 Billion cycles (**54.33× **slowdown)**.

This stark cliff illustrates the classic operating system phenomenon of **thrashing**: the system spends orders of magnitude more time stalling on I/O swaps than executing instructions.

7 Part V: Simulator Performance & High-Speed Optimization

Part V evaluates the wall-clock execution speed and computational throughput of the simulator across the benchmarks.

7.1 Optimization Strategies Implemented

To process trace files containing tens of millions of records within seconds, we incorporated several software optimizations:

1. **Bitwise Address Decomposition**: Rather than costly modulo and division operations, all VPN, offset, Level-1, and Level-2 extractions use single-cycle bitwise shifts and bitmasks:

```
uint32_t get_vpn() const { return address >> 12; }
uint32_t get_l1_index() const { return (address >> 22) & 0x03FF; }
uint32_t get_l2_index() const { return (address >> 12) & 0x03FF; }
```

2. $O(1)$ **TLB LRU Maintenance via Splice**: Rather than linearly searching an array or shifting vector elements upon every hit, the TLB uses `std::list::splice` on list iterators held in `std::unordered_map`:

```
bool TLB::lookup(uint32_t vpn, uint32_t& frame_out) {
    auto it = tlb_map.find(vpn);
    if (it != tlb_map.end()) {
        frame_out = it->second->frame;
        lru_list.splice(lru_list.begin(), lru_list, it->second); // O(1)
            move to MRU
        return true;
    }
    return false;
}
```

3. **Zero Heap Allocations on the Critical Path**: Once inner tables or TLB nodes are allocated, lookups, bit modifications, and hash updates require zero dynamic memory allocation (`new`/`malloc`).

4. **Optimized Compiler Toolchain**: Compiled with `-O2` optimization flags, enabling aggressive function inlining and loop vectorization.

7.2 Wall-Clock Execution Benchmark Profile

Table 6 summarizes the execution time and throughput on an x86-64 host machine.

Table 6: Part V Simulator Wall-Clock Performance and Throughput

Benchmark Trace	Total Trace Entries	Wall-Clock Time (s)	Throughput (Million Accesses/
`gcc`	12,473,306	2.51 s	**4.97 M accesses**
`mcf`	12,474,523	2.86 s	**4.36 M accesses**
`lbm`	12,843,431	2.77 s	**4.63 M accesses**
`x264`	11,721,344	2.28 s	**5.14 M accesses**

The simulator consistently achieves processing throughputs between **4.36 and 5.14 million memory accesses per second**, enabling multi-million-access trace files to be simulated in under three seconds.

8 Conclusions & Architectural Insights

This comprehensive laboratory investigation yields five overarching architectural conclusions:

1. **The Inefficiency of Flat Page Tables**: A linear single-level page table requires contiguous kernel memory for the entire address space ($1,048,576$ entries), yet typical benchmarks utilize fewer than $2,200$ pages. This results in an abysmal page table utilization of $< 0.2\%$, making flat page tables impractical for 32-bit and 64-bit systems.

2. **The Superiority of Hierarchical Multi-Level Tables**: Two-level hierarchical page tables with on-demand allocation reduce page table memory overhead by **93.07% to 99.32%**, enabling systems to support numerous processes concurrently without exhausting physical memory.

3. **The Synergistic Role of the TLB**: While multi-level page tables introduce extra memory lookup steps (400 vs. 200 cycles for page walks), a small 128-entry fully associative TLB achieves hit rates exceeding **99.4% to 99.9%**. This virtually eliminates the hierarchical walk penalty, yielding the optimal architectural balance: minimal memory overhead with single-cycle translation speed.

4. **Validation of Pareto Locality (Hypothesis B)**: Access distributions across virtual pages strictly follow a Pareto distribution, with the top 10% of pages capturing over 82% of memory references. This concentrated working set explains why a compact 128-entry TLB delivers such high hit rates.

5. **The Severe Cost of Thrashing**: When physical frame allocations drop below a process's active working set, disk fault penalties ($20,000,000$ cycles) trigger exponential execution slowdowns (over $54\times$ degradation), demonstrating the critical importance of effective working-set management in operating system schedulers.